Lex and Yacc Program to Parse the Grammar

Grammar

```
S -> L = R | R
L -> * R | id
R -> L
```

lexer.l

```
%{
#include "y.tab.h"
%}

%%

[a-zA-Z_][a-zA-Z0-9_]*   { return ID; }
[ \t]+                   ;
\n                       { return '\n'; }
.                        { return yytext[0]; }

%%

int yywrap() { return 1; }
```

parser.y

```
%{
#include <stdio.h>
#include <stdlib.h>
int yylex();
void yyerror(const char *s);
%}

%token ID

%%

program : program line
        |
        ;

line    : S '\n'     { printf("Valid string\n"); }
        ;

S       : L '=' R
        | R
        ;

L       : '*' R
        | ID
        ;

R       : L
        ;

%%

void yyerror(const char *s) {
    printf("Invalid string\n");
```

```
    exit(0);
}

int main() {
    printf("Enter a string (press Enter after each line, Ctrl+D to end):\n");
    yyparse();
    return 0;
}
```

How to run

```
lex lexer.l
yacc -d parser.y
gcc lex.yy.c y.tab.c -o parser
./parser
```

Sample inputs and outputs

```
id = id        ->  Valid string
*id = id       ->  Valid string
**a = *b       ->  Valid string
a              ->  Valid string
= a            ->  Invalid string
a = = b        ->  Invalid string
```